

Comparative Analysis of KCF, CSRT, and TLD for UAV Tracking, with a Proposed KCF–TLD Relocalization Strategy

Frank Xu

April 19, 2026

1 Introduction

Although many UAV visual tracking methods have been proposed, several classical discriminative trackers are still frequently used as reference points in both teaching and practice. Among them, KCF proposed by Henriques et al.^[9] relies on circulant matrices and the kernel trick so that correlation filtering can be solved efficiently in the Fourier domain. CSRT proposed by Lukežić et al.^[13] introduces channel reliability and spatial reliability in order to improve robustness while preserving the discriminative correlation-filter framework. TLD proposed by Kalal et al.^[10] combines tracking, learning, and detection, and updates its detector online so that the target may be found again after loss.

These three algorithms are often mentioned together as representative discriminative trackers for UAV-style visual tasks, yet the differences between them are far from minor. Their internal principles, performance priorities, runtime behaviour, and recovery ability are markedly different. For readers outside the tracking community, these differences are not always obvious from benchmark names or headline metrics alone. A tracker that is attractive because of its speed may become weak once the target is occluded. A tracker that is more robust under appearance change may impose a much heavier computational burden. A tracker with recovery ability may still be unsuitable as a constantly active main tracker.

For that reason, this paper first analyses KCF, CSRT, and TLD from the perspective

of algorithmic principle and engineering behaviour, with the aim of giving a clearer and more accessible comparison of their differences. It then uses that comparison to motivate a hybrid KCF–TLD design. The core idea is straightforward: KCF is extremely fast but weak once the target has been lost, whereas TLD is capable of redetection but much slower. Instead of treating them as interchangeable trackers, the proposed method lets each one perform the task to which it is best suited. KCF remains the main tracker during normal operation, and TLD is activated only when recovery becomes necessary.

2 Literature Review

Modern discriminative tracking research covers many variants and improvements, but for the present purpose it can be organised into several broad mechanism-oriented groups^[1]. One group is built around kernelized correlation filtering, represented here by KCF. A second group retains the correlation-filter foundation while improving reliability through feature or spatial weighting, represented here by CSRT. A third group places more emphasis on long-term recovery through the joint use of tracking, learning, and detection, represented here by TLD. Later studies extended these directions through scale adaptation, spatial regularisation, reliable patch design, complementary learning, and stronger appearance modelling^[2-3,5-8,11-12,16].

This review therefore concentrates on the three original algorithms that most directly correspond to those mechanisms. The purpose is not only to cite influential papers, but also to summarise what each original paper actually contributes and why those contributions matter for later hybrid design. The review also provides the conceptual background for the KCF–TLD strategy proposed in this report.

2.1 KCF: High-Speed Correlation Filtering

KCF learns a discriminative classifier by means of a kernelized correlation filter. The original paper by Henriques et al.^[9] shows that cyclic shifts of the target patch can be used to generate a large set of training samples efficiently, and that the resulting ridge-regression problem can be solved in the Fourier domain by exploiting the circulant structure of the sample matrix. This makes KCF important not only because it is fast in practice, but also because it gives a clear mathematical explanation for how high-speed discriminative

tracking can be achieved.

In its standard form, the regression objective can be written as

$$w = (X^T X + \lambda I)^{-1} X^T Y,$$

where X denotes the sample matrix, Y is the regression target, and λ is the regularisation parameter. By combining the kernel trick with the diagonalisation property of circulant matrices under the discrete Fourier transform, KCF greatly reduces computational complexity and can be implemented at very high frame rate.

Its strengths are therefore easy to understand. KCF is computationally efficient, and with FFT-based filtering it can sustain fast short-term tracking under ordinary conditions. The use of kernel mapping also helps it remain reasonably robust to pose change, illumination variation, and limited scale fluctuation. Its limitations are equally visible. KCF is much better at continuing to follow a target that is already visible than at finding a target again after loss. Severe occlusion, abrupt displacement, strong background interference, or accumulated drift can cause the tracker to fail, and once this happens the original KCF framework provides little direct support for relocalization. This weakness remains central even in later improved variants^[4,14-15,17].

2.2 CSRT: Reliability-Aware Correlation Filtering

Lukežić et al.^[13] proposed CSRT to show that the basic correlation-filter framework can be made more reliable by modelling channel reliability and spatial reliability. Rather than assigning the same weight to every feature channel and image region, CSRT estimates which channels carry useful information and which spatial areas are trustworthy, and then uses these weights during tracking. The original paper is important because it demonstrates that better localisation quality can be obtained without abandoning the discriminative correlation-filter family altogether.

In practice, the main advantage of CSRT is that its tracking precision and robustness are usually better than those of KCF, especially when target appearance changes, local occlusion occurs, or the object boundary is difficult to describe cleanly. It often produces tighter boxes and more stable localisation in challenging scenes. The cost is a substantial increase in computation. CSRT is slower than KCF because it uses richer features and

more careful reliability modelling. In other words, it improves stability by sacrificing part of the speed advantage that makes KCF attractive.

2.3 TLD: Long-Term Recovery Through Tracking, Learning, and Detection

TLD is composed of three linked parts: tracking, learning, and detection. In the original method of Kalal et al.^[10], the tracking branch is based on Median-Flow and uses optical-flow feature tracking together with forward-backward error to detect failure. The detection branch applies a sliding-window search with a cascade of classifiers, including a variance classifier, random ferns, and a nearest-neighbour classifier. The learning branch uses P-N learning to generate positive and negative samples online and update the detector accordingly.

The major advantage of TLD is explicit redetection. Compared with trackers that rely mainly on short-term local matching, TLD is more capable of finding the target again after occlusion, temporary disappearance, or recovery from drift. For that reason, it is theoretically better suited to post-failure recovery than KCF or CSRT. Its drawbacks are also prominent. TLD is computationally heavy, slower than KCF in practice, and sensitive to scene complexity. In cluttered environments it may introduce false detections or unstable recoveries. These properties make TLD more appropriate as a recovery module than as a constantly active high-speed main tracker.

2.4 Why a Hybrid KCF–TLD Design Is Needed

Although KCF, CSRT, and TLD differ significantly in mechanism, they still share several broader limitations. All three can become unstable under occlusion, rapid motion, large scale change, or strong background interference. None of them can easily satisfy high real-time speed, accurate localisation, and reliable post-failure recovery at the same time. They also rely mainly on short-term visual information, while their use of longer-term motion regularity and higher-level semantic context remains limited. These common constraints are exactly why a further hybrid design remains meaningful.

The KCF–TLD idea follows from this comparison. KCF is extremely fast but weak at recovery after loss. TLD has recovery ability but is too slow to serve as an efficient

main tracker throughout the whole sequence. The purpose of combining them is therefore not to stack two algorithms mechanically, but to let them perform different roles. KCF is used as the main tracker so that the system can retain the high frame rate brought by correlation filtering, while TLD is activated as an auxiliary recovery module when KCF becomes unstable or loses the target. In this sense, the hybrid is designed as a functional complement rather than a direct superposition.

3 Methodology

3.1 Overview of the Proposed Tracker

The proposed KCF–TLD tracker is designed around a clear division of labour. KCF is used as the main tracker during the normal stage so that the system can preserve high frame rate. TLD is not allowed to run continuously, because doing so would remove the main practical advantage of KCF. Instead, TLD acts as a recovery module and is activated only when KCF becomes unreliable or appears to have lost the target.

The algorithm begins from the initial bounding box on the first frame. KCF is initialized first, and the most recent reliable target box is stored as `last_bbox`. At the same time, the system records a failure counter, a pending recovery box, and a pending recovery confirmation counter. During most frames, the tracker simply runs KCF and checks whether the returned box is still reliable. If KCF remains stable, the result is accepted immediately and all recovery state is cleared.

Once KCF fails repeatedly or produces boxes that look unreliable, the algorithm enters recovery mode. TLD is then initialized from the last valid target state and is queried only on scheduled recovery frames. A candidate box proposed by TLD is not accepted at once. It must pass overlap, plausibility, and multi-frame consistency checks before it is treated as a valid relocalization result. After confirmation, KCF is reinitialized from that recovered box and the system returns to high-speed tracking. This design keeps most of the sequence in the fast KCF stage while still providing a chance to recover the target after failure.

3.2 KCF Stage

As the base tracker, KCF solves a regularised regression problem in the Fourier domain. Given a sample patch x and a Gaussian-shaped response y , the correlation filter can be written as

$$\hat{w} = \frac{\hat{x}^* \odot \hat{y}}{\hat{x}^* \odot \hat{x} + \lambda},$$

where $\hat{x}$ and $\hat{y}$ are the Fourier transforms of the training sample and desired response, $\odot$ denotes element-wise multiplication, and λ is the regularisation term. In each frame, KCF evaluates the response map in a local region around the previous target state and selects the position with the highest response as the new prediction. This keeps the main tracker computationally light and highly responsive.

In the implemented system, KCF is not treated as reliable by default. Its output is checked against the last valid box by simple geometric continuity rules. If the IoU with the previous valid box is too small, if the area changes too sharply, or if the box centre shifts too abruptly, the current KCF result is marked as unreliable. The algorithm also waits for repeated failure instead of reacting to one temporary fluctuation. In the present implementation, recovery mode is triggered when the failure streak reaches 2 frames. This keeps KCF responsible for fast local tracking while avoiding unnecessary recovery activation.

3.3 TLD Stage

The TLD branch is introduced in order to raise the probability of finding the target again after KCF failure. In the hybrid design, TLD is not expected to carry long-term high-speed tracking. Its role is narrower: it acts as a relocalization mechanism once the KCF stage has become unstable. This design choice follows directly from the strengths and weaknesses identified in the literature review.

To prevent TLD from dominating the total runtime, recovery mode is deliberately restricted. TLD is initialized only when recovery begins, and it is not updated on every frame thereafter. Instead, recovery is attempted only once every 3 frames in the current implementation. In addition, one recovery episode is limited to at most 120 frames. These restrictions are important because TLD is the main source of slowdown in difficult sequences. Without such limits, the hybrid would lose the speed advantage that motivates

the use of KCF in the first place.

3.4 Fusion Logic and Threshold Design

The fusion part is not designed as a forced blending of two trackers. Instead, it separates the working regions in which each tracker is most useful. KCF handles the main tracking process, whereas TLD is responsible for recovery after loss. The key practical issue is therefore how to decide that KCF is no longer stable enough and how to verify that a TLD recovery box is credible.

In the current system, KCF is treated as unstable when the continuity between the current box and the previous valid box becomes too weak. Two direct rules are used here. First, the current box is considered unreliable if its IoU with the previous valid box is lower than 0.15. Second, the area ratio between the current box and the previous valid box must remain between 0.5 and 2.0. These thresholds are simple heuristic rules, but they are easy to interpret and cheap to compute. Together with the failure streak threshold of 2 frames, they determine when TLD is allowed to enter recovery mode.

The next problem is how to judge whether the TLD recovery box is correct. The current design uses three checks. First, the candidate must overlap the last valid box by at least 0.3 IoU. Second, the candidate is not accepted after one hit; it must be confirmed across 2 consecutive valid recovery attempts. Third, these consecutive candidates must also be consistent with each other, with an IoU of at least 0.45. This multi-step confirmation is introduced to reduce the chance of accepting an incorrect recovery box too early.

The threshold design also reflects a strong awareness of runtime. Recovery mode is not allowed to call TLD continuously, and it is not allowed to continue indefinitely. The recovery interval is set to 3 frames, and the maximum recovery duration is 120 frames. The purpose is to keep the algorithm from remaining trapped in a slow TLD-dominated state for too long. In other words, the hybrid is designed not only to improve recovery, but also to keep the cost of recovery under control.

3.5 Flowchart of the Proposed Algorithm

Figure 1 summarises the overall logic of the proposed KCF–TLD tracker. A flowchart is more appropriate here than a long code listing, because the method is defined mainly by

decision paths and state transitions rather than by implementation syntax.

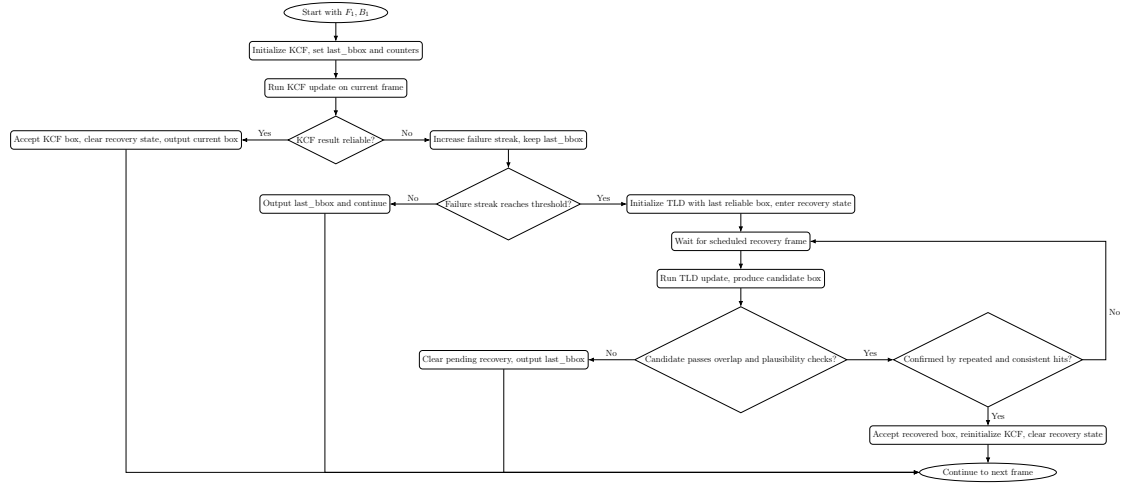


Figure 1: Overall flowchart of the proposed KCF-TLD relocalization tracker.

3.6 Recovery Decision Flow

Figure 2 isolates the recovery branch. This part is critical because the usefulness of the hybrid depends less on constant complexity than on whether recovery is triggered at sensible moments and whether doubtful TLD candidates are kept from being accepted too early.

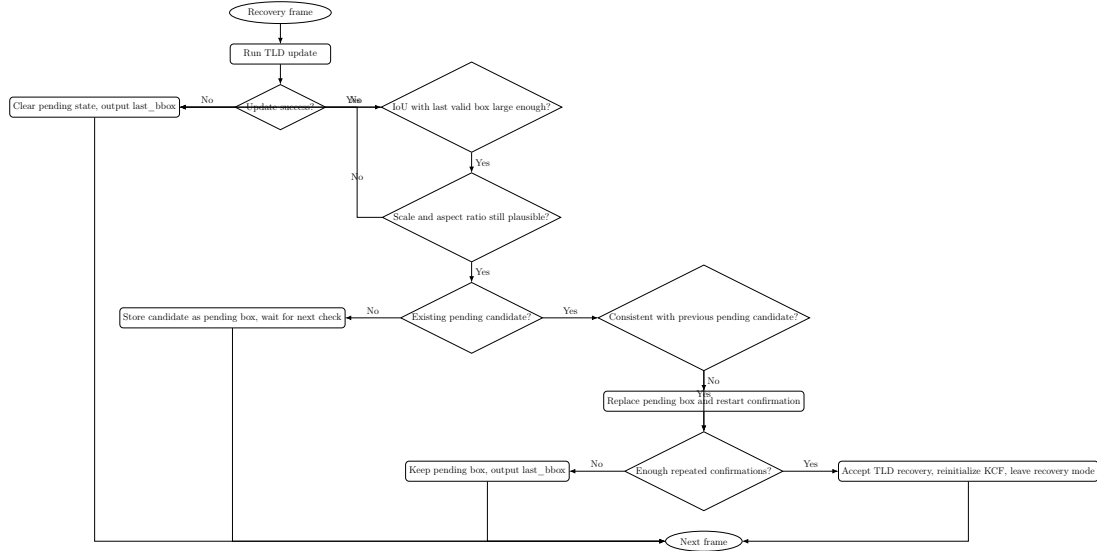


Figure 2: Detailed recovery decision flow for TLD candidate validation and KCF reinitialization.

4 Results and Analysis

4.1 Experimental Setup

The experiments were carried out on a directly comparable subset derived from UAV123. The tested sequences are *bike1* and *bike3*. To keep iterative debugging and validation manageable, the benchmark uses a frame step of 2, a maximum of 800 frames per sequence, and a resized evaluation resolution of 480×360 . The output includes IoU-related measures, localisation measures, stability indicators, efficiency indicators, diagnostic plots, and a leaderboard image.

It should be stated clearly that the validated benchmark in this report mainly compares KCF and the proposed KCF–TLD tracker under identical conditions. The earlier comparison among KCF, CSRT, and TLD is therefore used primarily to explain mechanism and design motivation, whereas the experiment section places greater emphasis on the proposed hybrid algorithm itself. This focus is intentional, because the main contribution of the report is not another descriptive review of three classical trackers, but the implementation and evaluation of the KCF–TLD strategy.

4.2 Per-Sequence Interpretation

The two tested sequences reveal two different aspects of the proposed design. On *bike1*, average IoU increases from 0.031499 to 0.051382, while FPS decreases from about 309 to 161. This result suggests that the recovery branch is not merely increasing complexity. It does improve tracking quality in at least part of the benchmark, although the improvement comes with a visible runtime cost.

On *bike3*, the result is much less favourable. Average IoU remains the same as KCF at 0.067584, but FPS drops sharply from about 337 to 15. This is an important reminder that the hybrid does not automatically improve every sequence. When TLD becomes active in a difficult scene, the computational burden may become very large without producing a corresponding gain in localisation.

From the perspective of algorithm analysis, this contrast is still meaningful. It shows that the main problem of the current hybrid is not the existence of state switching itself, but the cost of the TLD recovery stage. In easier or moderate situations, the additional recovery mechanism can provide useful protection. In harder scenes, however,

the same mechanism may dominate runtime and weaken overall practicality. The current KCF–TLD implementation should therefore be viewed as a functionally meaningful prototype rather than as a fully optimised final solution.

4.3 Aggregate Comparison

Table 1 condenses the same pattern into one view. Relative to KCF, the hybrid raises average IoU from 0.044169 to 0.057071, overlap AUC from 0.053110 to 0.065750, centre precision from 0.111650 to 0.142395, normalised precision from 0.187702 to 0.242718, and tracking availability from 0.033981 to 0.050162. The direction of change is therefore not confined to one measure. It appears in overlap quality, localisation quality, and the ability to remain on target.

That spread is important because it matches the intended role of the added recovery branch. If recovery had improved only one statistic while leaving the others unchanged, the gain would have been harder to interpret. Here the change is broader. The tracker behaves as though temporary loss is being handled more effectively, which is exactly the part of the problem that plain KCF leaves exposed.

The same table also prevents that improvement from being overstated. Average FPS falls from 319.160 to 109.669, so the hybrid does not preserve the main practical advantage of KCF. In UAV tracking this is not a minor side issue, since speed is part of whether a tracker remains usable at all. The aggregate result therefore supports a narrower claim: the present design improves tracking quality in a meaningful way, but it does so by paying a runtime cost that is still too high.

Table 1: Aggregate benchmark results on the current UAV123 subset run

Algorithm	Avg. IoU	AUC	Avg. CE	Prec.@20px	Norm. Prec.	Availability	Avg. FPS
KCF	0.044169	0.053110	83.531	0.111650	0.187702	0.033981	319.160
KCF–TLD	0.057071	0.065750	68.171	0.142395	0.242718	0.050162	109.669

4.4 Benchmark Figures

Figures 3, 4, and 5 present the cropped benchmark outputs from the latest validated run. The result images were trimmed and arranged in a two-column layout so that the comparison could be presented more compactly within the report.

Their function is analytical rather than decorative. The overlap and precision curves show whether the hybrid remains ahead across a range of thresholds instead of at one convenient operating point. The grouped metric panels separate overlap quality, localisation accuracy, stability, and efficiency, which makes the trade-off easier to read than in a single dense table. The leaderboard image then gives the same comparison in a compressed ranking format.

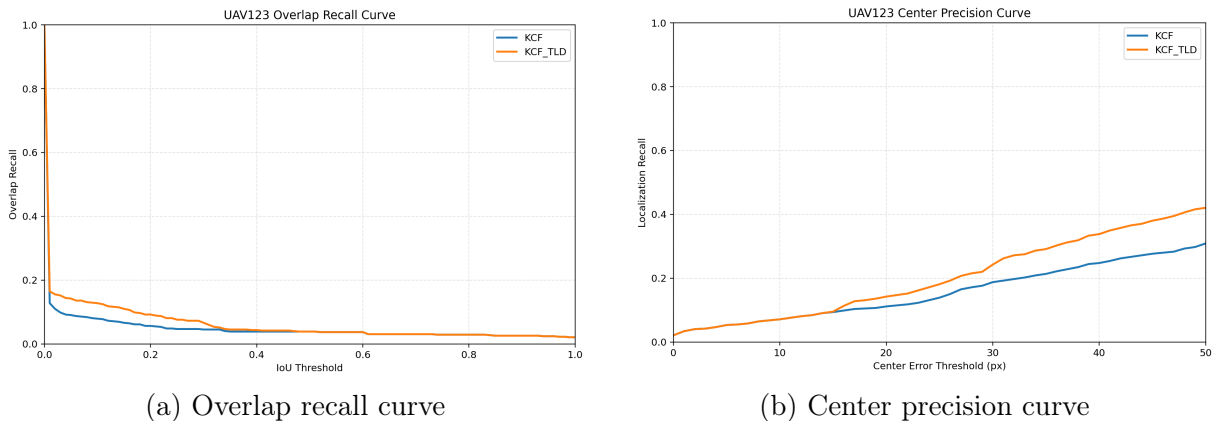


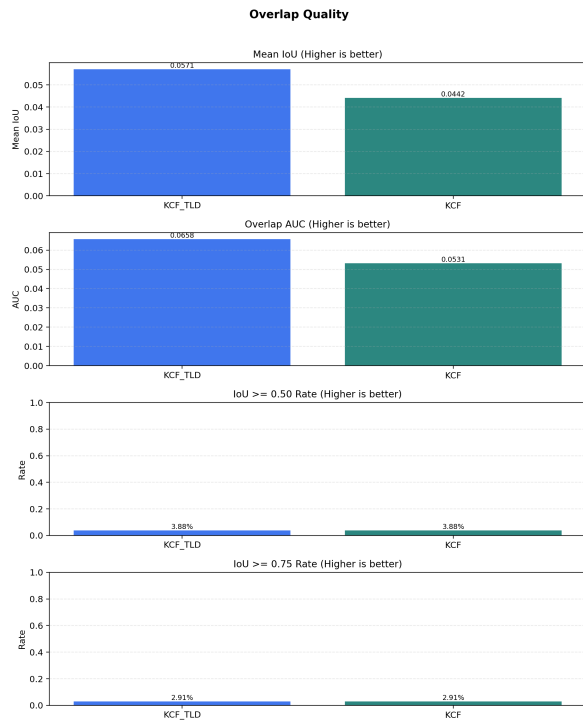
Figure 3: Paired benchmark curves for the current KCF versus KCF-TLD comparison.

The visual outputs reinforce the table rather than replacing it. In each view, the same balance reappears: quality tends to move in favour of the hybrid, while efficiency moves away from it. That repetition across several summaries makes the interpretation more credible, because the conclusion is not being drawn from one convenient metric alone.

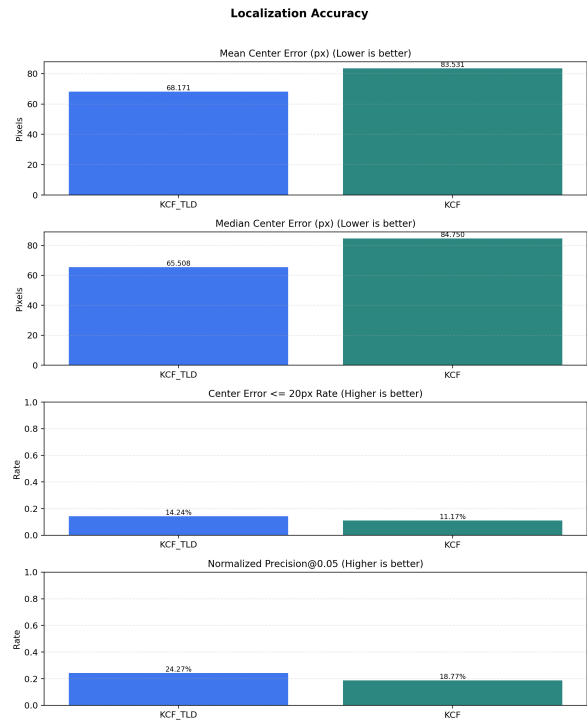
5 Discussion

The main advantage of the proposed method is that its design logic is easy to explain and closely tied to the strengths of the source algorithms. KCF is responsible for high-speed short-term tracking, while TLD is responsible for relocalization after failure. The two modules do not compete for the same task. Instead, they are assigned different functions. This makes the hybrid easier to interpret and tune in engineering terms.

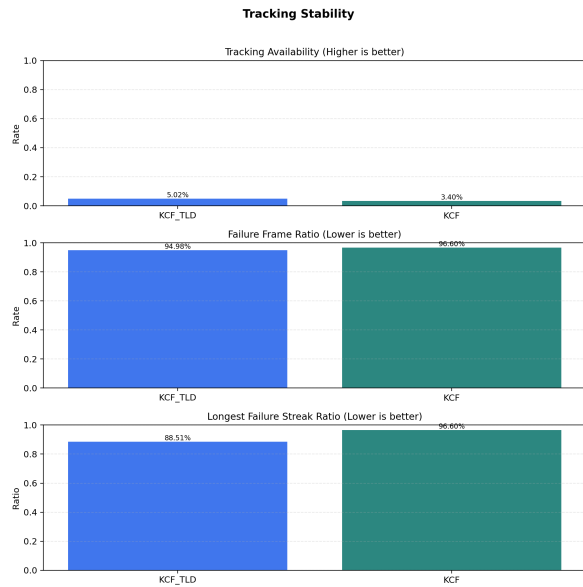
Relative to pure KCF, the hybrid also adds a more cautious protection mechanism. TLD is not triggered by one doubtful frame, and a recovery candidate is not accepted after one successful update. The recovery box must pass candidate filtering, consistency checking, and repeated confirmation before it is used to reinitialize KCF. These steps help reduce the chance of incorrect recovery. The method also shows a clear awareness of



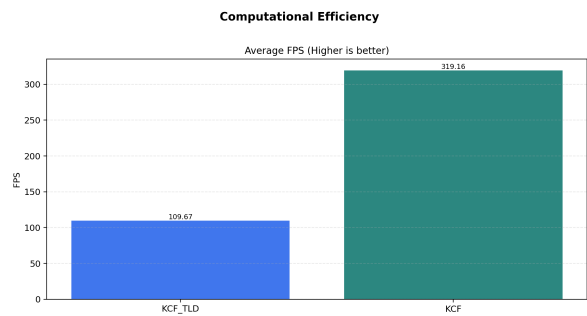
(a) Overlap quality metrics



(b) Localization accuracy metrics

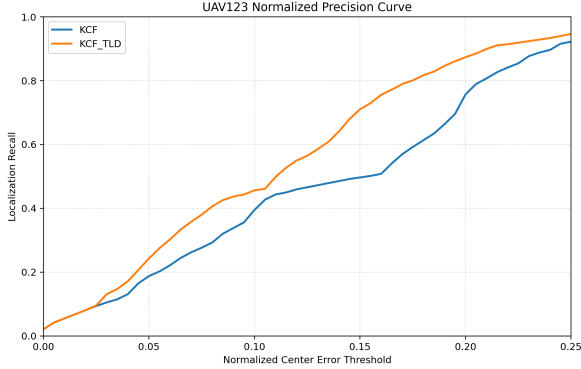


(c) Tracking stability metrics



(d) Computational efficiency metrics

Figure 4: Grouped benchmark metrics in a two-row layout. These panels correspond to the four evaluation categories used in the benchmark pipeline.



(a) Normalized precision curve

Benchmark Leaderboard			
Overlap Quality			
Metric	Direction	KCF_TLD	KCF
Mean IoU	Higher is better	0.071	0.042
Overlap AUC	Higher is better	0.008	0.011
IoU >= 0.50 Rate	Higher is better	3.8%	3.8%
IoU >= 0.75 Rate	Higher is better	2.9%	2.9%

Localization Accuracy			
Metric	Direction	KCF_TLD	KCF
Mean Center Error (px)	Lower is better	68.171	83.531
Median Center Error (px)	Lower is better	65.568	84.750
Center Error <= 20px Rate	Higher is better	14.24%	11.17%
Normalized Precision@0.5	Higher is better	29.27%	18.77%

Tracking Stability			
Metric	Direction	KCF_TLD	KCF
Tracking Availability	Higher is better	5.02%	3.46%
Failure Frame Ratio	Lower is better	94.98%	96.46%
Longest Failure Stride Ratio	Lower is better	88.51%	96.46%

Computational Efficiency			
Metric	Direction	KCF_TLD	KCF
Average FPS	Higher is better	109.67	119.16

(b) Benchmark leaderboard

Figure 5: Additional benchmark outputs showing normalized precision and the aggregate leaderboard.

speed, because recovery mode is limited by interval control and a maximum duration.

At the same time, the current method still has several clear limitations. First, the overall upper bound of the system is still strongly influenced by KCF, because ordinary tracking depends on KCF for most frames. Second, the failure judgment mechanism is based mainly on heuristic rules such as overlap, area change, and repeated failure. These rules are simple and interpretable, but their adaptability remains limited in scenes involving long occlusion, strong clutter, rapid motion, or strong non-linear change. Third, the recovery ability of TLD is not always stable, and the geometric constraints used here may also reduce the chance of successful recovery in large-range search situations.

Future improvement can be considered from two directions. The first concerns the trigger of recovery mode. The current decision is based mainly on fixed thresholds, which is convenient for implementation but still vulnerable to temporary disturbance. Later work could combine motion continuity, trajectory trend, response stability, and short temporal history to obtain a more intelligent estimate of tracker reliability. The second concerns recovery verification. At present, recovery is checked mainly by geometric consistency. In the future, stronger appearance cues, temporal context, or even large-model-based reasoning about motion direction and appearance continuity may help reduce false recovery and improve robustness in more complex UAV scenes.

6 Conclusion

This paper examined KCF, CSRT, and TLD as three representative discriminative tracking algorithms for UAV-style visual tracking tasks, and compared them from the perspective of principle, behaviour, and practical use. The comparison shows that KCF is attractive because of its very high speed, CSRT is stronger in localisation reliability and robustness, and TLD is more valuable in the stage after failure because of its explicit redetection ability. These differences explain why no single one of the three methods can easily satisfy speed, accuracy, and recovery at the same time.

On that basis, the paper proposed a KCF–TLD hybrid tracker. The method uses KCF to preserve high-speed tracking during the normal stage, and activates TLD only when KCF becomes unstable or loses the target. A multi-frame confirmation mechanism is then used to make the accepted recovery box more reliable before KCF is reinitialized. The experiments indicate that this “high-speed main tracker plus auxiliary recovery module” design is feasible and can improve several quality-related measures relative to pure KCF, although it also exposes a clear efficiency cost when recovery becomes expensive.

The present KCF–TLD tracker should therefore be regarded as a meaningful attempt rather than a finished final answer. Its current performance still depends heavily on heuristic thresholds and hand-crafted decision rules. Future work can improve the method in a more intelligent, adaptive, and multimodal direction. More refined state evaluation may make recovery triggering more accurate, while stronger modelling of motion continuity, appearance consistency, and scene context may make recovery verification more robust. If these directions can be implemented effectively, the algorithm may achieve stronger stability, recovery ability, and overall tracking performance in complex UAV scenarios.

References

- [1] BAI H. Shendu Dan Mubiao Zongsuanfa Zongshu [Review of Deep Single-Object Tracking Algorithms][J]. *Xiandai Jisuanji*, 2021(9): 113-116.
- [2] BERTINETTO L, VALMADRE J, GOLODETZ S, MIKSIK O, TORR P H S. Staple: Complementary Learners for Real-Time Tracking[C/OL]//*Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016: 1401-1409.

- [3] BIBI A, GHANEM B. Multi-Template Scale-Adaptive Kernelized Correlation Filters[C/OL]//Proceedings of the IEEE International Conference on Computer Vision (ICCV) Workshops. 2015: 613-620.
- [4] CHEN X, YANG Z, YU W. An Improved KCF Mixed with Deep Feature for Pedestrian Tracking[J/OL]. Journal of Physics: Conference Series, 2023, 2465(1): 012015.
- [5] DANELLJAN M, BHAT G, KHAN F S, FELSBURG M. ATOM: Accurate Tracking by Overlap Maximization[C/OL]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2019: 4660-4669.
- [6] DANELLJAN M, HAGER G, KHAN F S, FELSBURG M. Learning Spatially Regularized Correlation Filters for Visual Tracking[C/OL]//Proceedings of the IEEE International Conference on Computer Vision (ICCV). 2015: 4310-4318.
- [7] GAO J, ZHANG T, YANG X, XU C. Deep Relative Tracking[J/OL]. IEEE Transactions on Image Processing, 2019, 28(12): 5987-6002.
- [8] GUO Q, FENG W, ZHOU C, HUANG R, WAN L, WANG S. Learning Dynamic Siamese Network for Visual Object Tracking[C/OL]//Proceedings of the IEEE International Conference on Computer Vision (ICCV). 2017: 1781-1789.
- [9] HENRIQUES J F, CASEIRO R, MARTINS P, BATISTA J. High-Speed Tracking with Kernelized Correlation Filters[J/OL]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2015, 37(3): 583-596.
- [10] KALAL Z, MIKOLAJCZYK K, MATAS J. Tracking-Learning-Detection[J/OL]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2012, 34(7): 1409-1422.
- [11] KRISTAN M, MATAS J, LEONARDIS A, et al. The Seventh Visual Object Tracking VOT2019 Challenge Results[C/OL]//Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops. 2019: 2206-2241.
- [12] LI Y, ZHU J, HOI S C H. Reliable Patch Trackers: Robust Visual Tracking by Exploiting Reliable Patches[C/OL]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2015: 353-361.
- [13] LUKEŽIĆ A, VOJÍŘ T, ZAJC L Č, MATAS J, KRISTAN M. Discriminative Correlation Filter with Channel and Spatial Reliability[C/OL]//2017 IEEE Conference

- on Computer Vision and Pattern Recognition (CVPR). 2017: 4847-4856.
- [14] TANG D, JIN W, LIU D, CHE J, YANG Y. Siam Deep Feature KCF Method and Experimental Study for Pedestrian Tracking[J/OL]. Electronics, 2023, 12(3): 669.
 - [15] WANG C, ZHANG L, XIE L, YUAN J. Kernelized Correlation Filter Tracking with Scale Adaptation and Spatial Reliability[J/OL]. IEEE Access, 2018, 6: 50748-50758.
 - [16] ZHANG K, ZHANG L, LIU Q, ZHANG D, YANG M H. Fast Visual Tracking via Dense Spatio-Temporal Context Learning[C/OL]//Proceedings of the European Conference on Computer Vision (ECCV). 2014: 127-141.
 - [17] ZHANG L, SUGANTHAN P N. Visual Tracking with Convolutional Neural Network and Kernelized Correlation Filters[J/OL]. Journal of Visual Communication and Image Representation, 2017, 45: 174-183.